Artificial Intelligence

AI Assignment | Release 28 September 2026 | Deadline 31 October 2026

Development of a Classical AI Reasoning Agent for Natural-Language Entailment

Symbolic inference over RuleTaker, FOLIO and CounterBench

Student	Mayuri Rajesh Pujari
Roll Number	B23ES1026
Instructor	Prof. Deeksha Varshney
Course	Artificial Intelligence
Assignment	Classical Reasoning Agent (Entailment: True / False)
Language	Python 3.12
Source code	Mayu-infinite/classical-ai-reasoning-agent

Contents

1 Abstract **2**

2 Introduction **2**
2.1 Problem Statement . . . 2
2.2 Objectives . . . 2

3 Datasets **3**

4 System Architecture **3**
4.1 Modular Implementation . . . 4

5 Knowledge Representation **4**

6 Classical Reasoning Algorithms **5**
6.1 Forward Chaining . . . 5
6.2 Backward Chaining . . . 5
6.3 Unification . . . 5
6.4 Resolution Refutation . . . 5
6.5 Closed-World and Open-World Semantics . . . 5

7 Experimental Setup **6**

8 Results **6**
8.1 Main Evaluation . . . 6
8.2 RuleTaker by Reasoning Depth . . . 7
8.3 FOLIO . . . 7
8.4 CounterBench . . . 8

9 Ablation Study **8**
9.1 Forward versus Backward Chaining . . . 8
9.2 Closed-World versus Open-World Reasoning . . . 9
9.3 Resolution Time Budget . . . 9
9.4 First-Order-Logic Preprocessing . . . 9
9.5 Natural-Language-Only FOLIO . . . 9

10 Reasoning Traces **9**

11 Limitations and Discussion **10**

12 Conclusion and Future Work **11**

13 Source Code Repository **11**

14 References **11**

1 Abstract

This project develops a general-purpose reasoning system that decides whether a statement is logically entailed by a knowledge base of facts and rules. The system follows a modular pipeline: natural-language processing, logical representation, selection of a reasoning engine, symbolic inference, and prediction. The final decision is always taken by a classical reasoning engine (Datalog-style forward and backward chaining, unification, resolution refutation, and a dedicated causal module); no generative model is used to produce the answer.

The system is evaluated on three public benchmarks: RuleTaker, FOLIO and CounterBench. The recorded experiments give **100.00%** accuracy on the RuleTaker test split (151,911 examples), **87.68%** on FOLIO validation, **81.82%** on FOLIO training data, and **95.00%** on CounterBench, for an aggregate of **99.83%** over 154,315 examples in 71.4 seconds. Ablations compare forward and backward chaining, closed- and open-world semantics, resolution time budgets, and first-order-logic preprocessing, and show that the choice of reasoning strategy and the quality of the logical representation matter as much as the inference engine itself.

Keywords: classical AI, knowledge representation, logical entailment, Datalog, forward chaining, backward chaining, resolution, first-order logic, causal reasoning.

Source Code

Mayu-infinite/classical-ai-reasoning-agent

Full implementation (`src/reasoning_agent/`), experiments, tests and the submission notebook:
https://github.com/Mayu-infinite/classical-ai-reasoning-agent

2 Introduction

2.1 Problem Statement

Given a knowledge base (KB) of facts and rules and a query, the system must output

$$\text{Prediction} \in \{\text{True}, \text{False}\},$$

indicating whether the query is entailed by the KB. The pipeline is

$$\text{Natural Language} \rightarrow \text{Internal Representation} \rightarrow \text{Classical Reasoning} \rightarrow \text{Prediction}.$$

The conversion from language to representation may use any computational method, but the final decision must come from a classical reasoning engine written for this project.

Running example (from the assignment)

KB: Bob is young. Bob is kind. If someone is young, then they are smart. If someone is smart and kind, then they are successful.
Query: Bob is successful. **Answer:** True, since $Young(Bob) \rightarrow Smart(Bob)$ and $Smart(Bob) \wedge Kind(Bob) \rightarrow Successful(Bob)$.

2.2 Objectives

1. Build a modular system that processes natural-language knowledge bases and queries.
2. Convert statements into representations suitable for symbolic reasoning.

3. Implement and integrate classical reasoning procedures.
4. Support logical (rule-based, first-order) and causal (counterfactual) reasoning.
5. Evaluate on RuleTaker, FOLIO and CounterBench.
6. Analyse how reasoning strategy and preprocessing choices affect accuracy and efficiency.
7. Inspect reasoning traces to see how conclusions are obtained.

3 Datasets

Table 1: Public benchmarks used for development and evaluation.

Dataset	Evaluates	Recorded evaluation
RuleTaker	Rule-based logical reasoning over facts and rules, grouped by proof depth	Development (ablations) and test split; test results broken down by depth group
FOLIO	Natural-language reasoning with first-order logic (quantifiers, predicates, connectives)	Training and validation splits; per-label results; preprocessing and time-budget ablations
CounterBench	Counterfactual reasoning	1,200 questions in five categories: basic, conditional, joint, nested, backdoor

Only public data were used for development and evaluation. No private test examples and no manually produced private-test predictions appear in any reported result.

4 System Architecture

The system separates language processing and logical representation from inference. Figure 1 gives the conceptual picture of symbolic reasoning (knowledge base → inference engine → derived conclusion), and Figure 2 shows the concrete pipeline of this implementation.

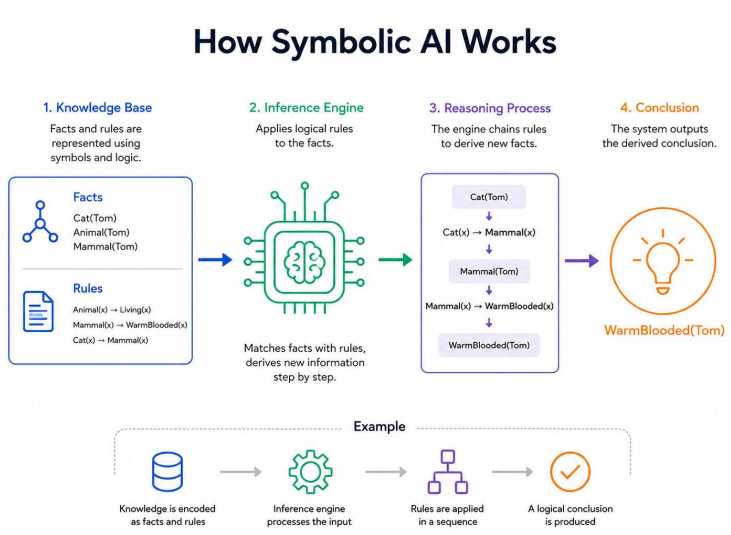


Figure 1: Conceptual view of symbolic reasoning: facts and rules are stored in a knowledge base, an inference engine chains the rules, and a conclusion is derived.

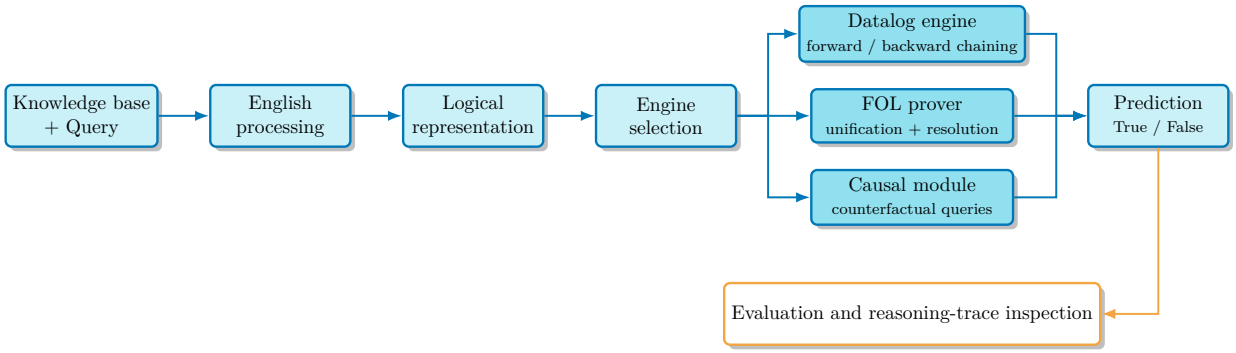


Figure 2: Pipeline of the implemented system: the logical representation is routed to the Datalog, first-order or causal engine, which produces the final prediction.

4.1 Modular Implementation

The active implementation lives in `src/reasoning_agent/`.

Table 2: Modules of the reasoning agent.

Module	Function
`agent.py`	Main interface; coordinates parsing, engine selection and prediction
`english.py`	Natural-language processing of knowledge bases and queries
`logic.py`	Logical representation (terms, atoms, rules, clauses) and operations such as unification
`datalog.py`	Datalog-style rule reasoning (forward and backward chaining)
`prover.py`	Proof procedures (resolution refutation)
`causal.py`	Causal reasoning for counterfactual questions
`data.py`	Dataset loading and utilities
`evaluate.py`	Accuracy and timing evaluation

- **Experiments** (`experiments/`): `evaluate_all.py`, `ablations.py`, `make_figures.py`, `save_results.py`, `show_traces.py`, `inspect_datasets.py`, `preview_datasets.py`.
- **Tests** (`tests/`): agent, causal reasoning, Datalog reasoning, English processing, and logic/proof procedures; run with `uv run pytest -q`.
- **Submission**: `submission/reasoning_agent.ipynb` generates `submission/submission.csv`.

5 Knowledge Representation

The reasoning engines operate on structured logical objects, not on raw sentences.

Facts and rules. A fact is directly available information, e.g. $Young(Bob)$. A rule is an implication, e.g. $Young(x) \rightarrow Smart(x)$ or $Smart(x) \wedge Kind(x) \rightarrow Successful(x)$.

Horn-style rules. Rule-based inference uses Horn rules

$$A_1 \wedge A_2 \wedge \cdots \wedge A_n \rightarrow B,$$

with premises A_i and a single conclusion B, for example

$$Parent(x, y) \wedge Parent(y, z) \rightarrow Grandparent(x, z).$$

This is the representation used by the Datalog engine.

First-order logic. FOLIO requires predicates, variables, quantifiers and connectives, e.g. $\forall x\,(Human(x) \rightarrow Mortal(x))$ together with $Human(Socrates)$ entails $Mortal(Socrates)$.

Clauses. Resolution works on clause form. A formula is converted to a disjunction of literals, using Skolemisation where existential quantifiers occur; for example $P(x) \rightarrow Q(x)$ becomes $\neg P(x) \vee Q(x)$.

Causal representation. Counterfactual questions (CounterBench) are handled by the separate causal module, whose results are reported by question category: basic, conditional, joint, nested and backdoor.

6 Classical Reasoning Algorithms

6.1 Forward Chaining

Forward chaining is data-driven: starting from the known facts it repeatedly fires rules whose premises hold until nothing new can be derived (a fixpoint). With facts A, C and rules $A \rightarrow B$, $B \wedge C \rightarrow D$ it derives B, then D, so $KB \models D$. It is effective when many conclusions can be derived from the same facts. In the experiments, negation is handled with well-founded semantics.

6.2 Backward Chaining

Backward chaining is goal-directed: to prove D it finds a rule concluding D and recursively proves that rule's premises, stopping at facts. It avoids deriving unrelated conclusions. The implementation uses SLDNF resolution with tabling (memoising sub-goals, which also prevents infinite loops on recursive rules).

6.3 Unification

Unification finds a substitution that makes two expressions identical. For $Parent(x, Bob)$ and $Parent(Alice, y)$ the most general unifier is $\theta = \{x/Alice,\ y/Bob\}$, giving $Parent(Alice, Bob)$. It is used to match rule premises with facts and to resolve clauses with variables.

6.4 Resolution Refutation

To show $KB \models Q$, the KB is converted to clauses, the negated query $\neg Q$ is added, and resolution is applied until the empty clause is derived:

$$KB \models Q \iff KB \wedge \neg Q \text{ is unsatisfiable.}$$

Because proof search can be expensive, each proof is run with a per-proof time limit, studied in Section 9.3.

6.5 Closed-World and Open-World Semantics

Under the *closed-world* assumption (negation as failure), anything not derivable is false. Under the *open-world* assumption, absence of a statement does not imply its negation, and reasoning uses classical negation with resolution. Both are compared in Section 9.2.

7 Experimental Setup

Metrics. Accuracy is the percentage of correct predictions. Efficiency is measured by total execution time (seconds) and mean inference time per query,

$$T_{avg(ms)} = \frac{\text{total time (s)}}{\text{number of queries}} \times 1000.$$

Environment. The saved run metadata is summarised in Table 3.

Table 3: Recorded run metadata.

Parameter	Value
Python version	3.12.13
Operating system / architecture	Linux, x86_64
RuleTaker split	test
Ablation experiments	enabled
Recorded wall-clock time	322.3 s

8 Results

8.1 Main Evaluation

Table 4: Accuracy and efficiency on the public datasets.

Dataset	Correct	Total	Accuracy	Time (s)	ms/query
RuleTaker test	151,911	151,911	100.00%	17.339	0.1141
FOLIO validation	178	203	87.68%	11.659	57.4321
FOLIO train	819	1,001	81.82%	42.218	42.1757
CounterBench	1,140	1,200	95.00%	0.172	0.1432
Overall	**154,048**	**154,315**	**99.83%**	**71.387**	**0.4626**

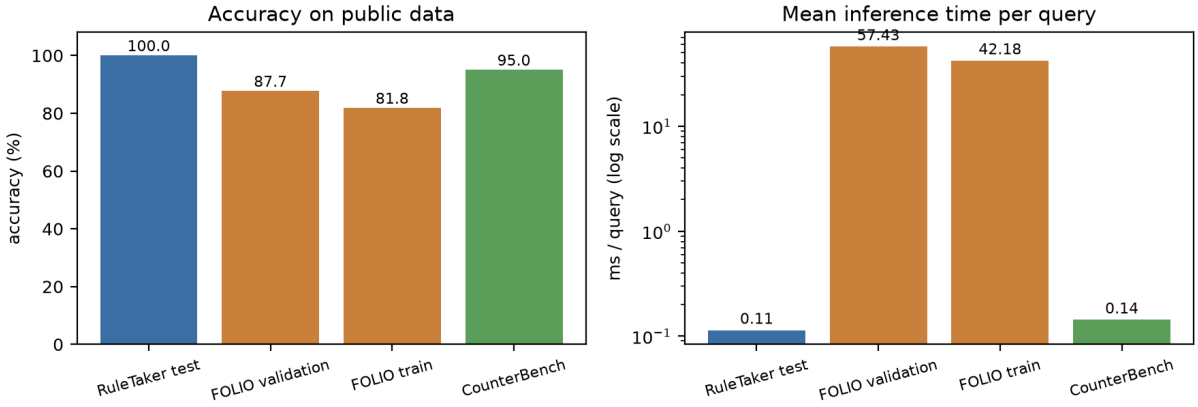


Figure 3: Accuracy (left) and mean inference time per query on a log scale (right) for each dataset.

RuleTaker is solved perfectly and almost instantly (about 0.11 ms per query) because rule-based Datalog inference on already-structured contexts is cheap. FOLIO is slower by more than two

orders of magnitude (42–57 ms per query), reflecting the cost of first-order resolution. The overall 99.83% is dominated by the very large RuleTaker test set, so the per-dataset results are the more informative numbers.

8.2 RuleTaker by Reasoning Depth

Table 5: RuleTaker test accuracy by reasoning group.

Group	Correct	Total	Acc.	Group	Correct	Total	Acc.
NatLang	7,793	7,793	100%	depth-3ext	23,222	23,222	100%
depth-0	17,347	17,347	100%	depth-3ext-NatLang	31,017	31,017	100%
depth-1	17,788	17,788	100%	depth-5	19,021	19,021	100%
depth-2	17,741	17,741	100%				
depth-3	17,982	17,982	100%				

Accuracy is 100% in every group, including depth-5 problems that need five chained inferences and the natural-language (NatLang) variants, so performance does not degrade with proof depth.

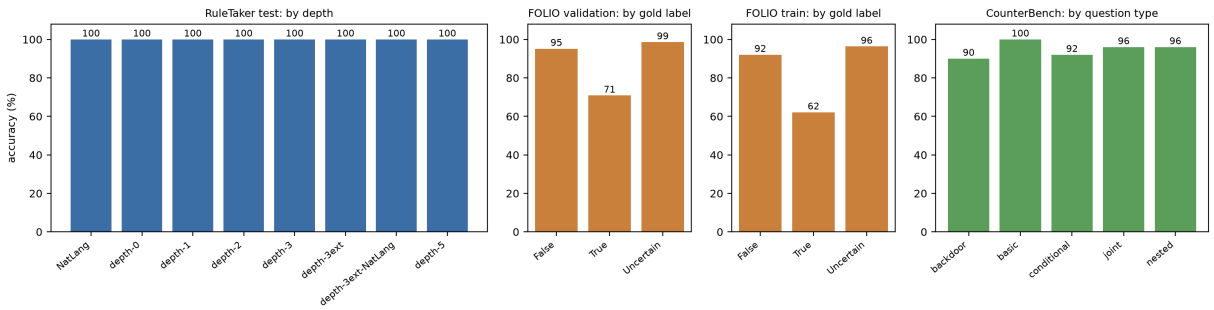


Figure 4: Accuracy breakdown: RuleTaker by depth, FOLIO by gold label (validation and train), and CounterBench by question type.

8.3 FOLIO

Table 6: FOLIO per-label accuracy.

	Validation			Train		
Label	Correct	Total	Acc.	Correct	Total	Acc.
False	59	62	95.16%	266	289	92.04%
True	51	72	70.83%	241	388	62.11%
Uncertain	68	69	98.55%	312	324	96.30%

In both splits the True class is clearly the hardest (70.83% and 62.11%), while False and Uncertain exceed 92%. A True label requires a complete proof to be found, whereas False and Uncertain can be reached when no proof (or no proof of the negation) exists; any incomplete or imperfect logical form therefore hurts True questions most.

8.4 CounterBench

Table 7: CounterBench accuracy by question type.

Question type	Correct	Total	Accuracy
Basic	250	250	100.00%
Joint	240	250	96.00%
Nested	240	250	96.00%
Conditional	230	250	92.00%
Backdoor	180	200	90.00%

Basic questions (a single change to one cause) are solved perfectly; accuracy falls as questions involve observed conditions (conditional) or confounding paths (backdoor), the lowest category at 90%.

9 Ablation Study

Figure 5 summarises all ablations; the following subsections discuss them in turn.

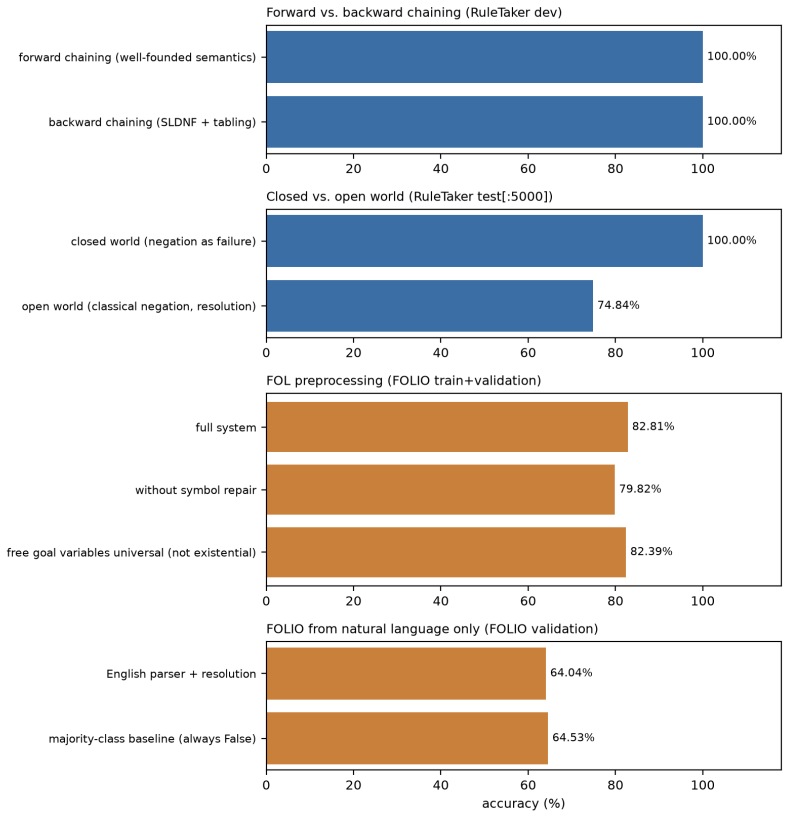


Figure 5: Ablation results: chaining strategy, closed vs. open world, FOL preprocessing, and natural-language-only FOLIO.

9.1 Forward versus Backward Chaining

Both strategies reach 100% on 75,872 RuleTaker development examples. Forward chaining (well-founded semantics) takes 7.722 s and backward chaining (SLDNF with tabling) 6.015 s, so backward

chaining is about 22% faster at equal accuracy, because it only explores rules relevant to the query.

9.2 Closed-World versus Open-World Reasoning

On the first 5,000 RuleTaker test examples, closed-world reasoning (negation as failure) scores 100% in 0.737 s, while open-world reasoning (classical negation with resolution) scores 74.84% in 1.257 s. The RuleTaker labels follow closed-world semantics, so the open-world reading leaves many queries undecided; the assumption used to interpret missing information must match the dataset.

9.3 Resolution Time Budget

On FOLIO validation, four per-proof time limits were tested. Raising the limit from 0.05 s to 0.1 s improves accuracy from 86.70% to 87.68%; raising it further to 0.3 s and 1.0 s gives no further gain, while total time grows from roughly 4 s to roughly 32 s (Figure 6). A budget of 0.1 s is therefore the best accuracy/time trade-off.

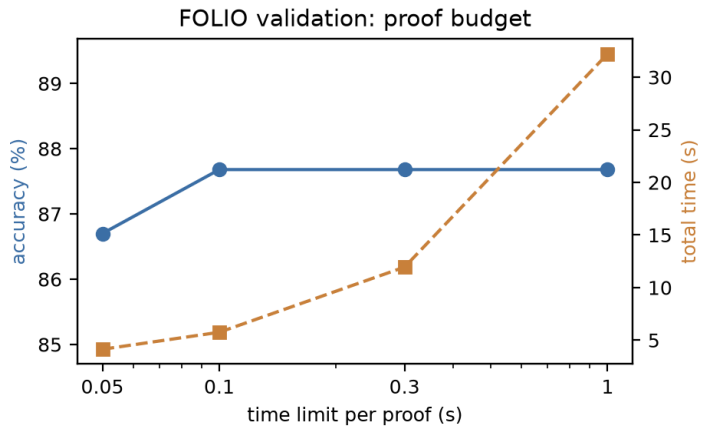


Figure 6: FOLIO validation: accuracy (solid) and total time (dashed) as the per-proof time limit grows.

9.4 First-Order-Logic Preprocessing

On 1,204 FOLIO training and validation examples the full pipeline scores 82.81%. Removing symbol repair lowers accuracy to 79.82% (about 3 points), and interpreting free goal variables as universal instead of existential gives 82.39%. Preprocessing choices therefore directly affect the quality of the logical forms and the resulting proofs.

9.5 Natural-Language-Only FOLIO

Using the English parser plus resolution (no gold logical forms) gives 64.04% on FOLIO validation, slightly below the always-False majority baseline of 64.53%. Translating unrestricted English into logic is the main bottleneck of the whole pipeline; the gap to the FOLIO numbers in Table 4 indicates that those results depend on first-order forms rather than raw English.

10 Reasoning Traces

The traces below show how each engine decides the running example. They are step-by-step derivations of the assignment example in the form the engines follow; the exact output of the implementation for any query can be printed with `experiments/show_traces.py`.

Forward chaining (data-driven)

Round 0	Facts: $Young(Bob)$, $Kind(Bob)$
Round 1	Rule $Young(x) \rightarrow Smart(x)$ with $\theta = \{x/Bob\}$ fires: derive $Smart(Bob)$
Round 2	Rule $Smart(x) \wedge Kind(x) \rightarrow Successful(x)$ with $\theta = \{x/Bob\}$ fires: derive $Successful(Bob)$
Round 3	No new facts (fixpoint). Query $Successful(Bob)$ is in the derived set $\Rightarrow$ **True**

Backward chaining (goal-directed, SLD)

Goal	$Successful(Bob)$
Step 1	Rule $Smart(x) \wedge Kind(x) \rightarrow Successful(x)$, $\theta = \{x/Bob\}$: sub-goals $Smart(Bob)$, $Kind(Bob)$
Step 2	$Smart(Bob)$: rule $Young(x) \rightarrow Smart(x)$: sub-goal $Young(Bob)$ $\Rightarrow$ matches a fact ✓
Step 3	$Kind(Bob)$ $\Rightarrow$ matches a fact ✓
Result	All sub-goals proved $\Rightarrow$ **True**

Resolution refutation (first-order prover)

Clauses	C1: $\neg Young(x) \vee Smart(x)$ C2: $\neg Smart(x) \vee \neg Kind(x) \vee Successful(x)$ C3: $Young(Bob)$ C4: $Kind(Bob)$
Negated goal	C5: $\neg Successful(Bob)$
Step 1	C5 + C2, $\theta = \{x/Bob\}$: $\neg Smart(Bob) \vee \neg Kind(Bob)$
Step 2	with C4: $\neg Smart(Bob)$
Step 3	with C1, $\theta = \{x/Bob\}$: $\neg Young(Bob)$
Step 4	with C3: empty clause $\square$ $\Rightarrow$ $KB \wedge \neg Q$ unsatisfiable $\Rightarrow$ **True**

All three engines agree on $KB \models Successful(Bob)$. The traces also show the practical difference between the strategies: forward chaining derives every consequence, backward chaining touches only the rules needed for the query, and resolution proves the goal by contradiction, which is what allows it to handle the richer first-order statements of FOLIO.

11 Limitations and Discussion

- **Natural-language parsing.** With English-only input FOLIO accuracy is 64.04%, so unrestricted sentence-to-logic translation remains the weakest stage.
- **First-order complexity.** Resolution can blow up on complex clause sets; the time-budget study shows accuracy saturating while cost keeps rising.
- **Dataset-specific weak spots.** FOLIO's True class (62–71%) and CounterBench's backdoor questions (90%) are the hardest categories.
- **Aggregate accuracy.** The 99.83% overall figure is dominated by the 151,911 RuleTaker examples and should be read together with the per-dataset results.

- **Generalisation.** Results are on public benchmarks and do not guarantee identical performance on the unseen private test set.

12 Conclusion and Future Work

The project delivers a modular classical-AI reasoning system in which every prediction is produced by an explicit symbolic engine. It reaches 100.00% on RuleTaker test, 87.68% on FOLIO validation, 81.82% on FOLIO train and 95.00% on CounterBench, at a mean of 0.46 ms per query overall. The ablations show that backward chaining is faster than forward chaining at equal accuracy, that closed-world semantics must match the dataset (100% vs. 74.84%), that a 0.1 s proof budget suffices, and that symbol repair is worth about three accuracy points. Future work: a stronger English-to-logic translator, support for more complex quantifier structures, more efficient resolution search, richer causal dependencies, and clearer trace presentation.

13 Source Code Repository

Mayu-infinite/classical-ai-reasoning-agent

https://github.com/Mayu-infinite/classical-ai-reasoning-agent

The repository contains the reasoning-agent package (`src/reasoning_agent/`), the experiment and figure scripts (`experiments/`), the test suite (`tests/`), and the submission notebook (`submission/reasoning_agent.ipynb`) that regenerates `submission.csv`.

14 References

1. P. Clark, O. Tafjord, and K. Richardson, "Transformers as Soft Reasoners over Language," in *Proc. IJCAI*, 2020. (RuleTaker)
2. S. Han *et al.*, "FOLIO: Natural Language Reasoning with First-Order Logic," arXiv:2209.00840, 2022.
3. Y. Chen, V. K. Singh, J. Ma, and R. Tang, "CounterBench: Evaluating and Improving Counterfactual Reasoning in Large Language Models," *Proc. AAAI*, vol. 40, no. 36, pp. 30350–30358, 2026; arXiv:2502.11008. Dataset: huggingface.co/datasets/CounterBench/CounterBench.
4. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2020.
5. J. A. Robinson, "A Machine-Oriented Logic Based on the Resolution Principle," *Journal of the ACM*, vol. 12, no. 1, pp. 23–41, 1965.
6. A. Martelli and U. Montanari, "An Efficient Unification Algorithm," *ACM Trans. Programming Languages and Systems*, vol. 4, no. 2, pp. 258–282, 1982.
7. K. L. Clark, "Negation as Failure," in *Logic and Data Bases*, Plenum Press, 1978, pp. 293–322.
8. A. Van Gelder, K. A. Ross, and J. S. Schlipf, "The Well-Founded Semantics for General Logic Programs," *Journal of the ACM*, vol. 38, no. 3, pp. 619–649, 1991.

9. W. Chen and D. S. Warren, "Tabled Evaluation with Delaying for General Logic Programs," *Journal of the ACM*, vol. 43, no. 1, pp. 20–74, 1996.

10. AI Assignment specification, released 28 September 2026.